SituationReport

testdata_generator

User Manual

Demo data rooms and synthetic Jira data for SituationReport

---

situation-report -- github.com/Jaegerfeld/situation-report

For Agile Coaches, PI Managers and beta testers -- Version 0.54.0

1 What is the Testdata Generator?

The **Testdata Generator** produces invented but realistic data for SituationReport — so you can demonstrate, learn and test the portfolio report, delta briefing, conference folder and the registers without touching real project data. No persons appear; the same seed yields the same data room.

The generator has two paths. The **data room** is the default path of the beta: in three steps it builds a complete portfolio folder with structure, story, Jira exports per ART, registers, provider raw data, a series of stands and the rendered delta briefings. The **single ART** is the classic path: one Jira export file from a workflow file, with every knob for cycle time and flow patterns (chapter 8).

> **Which path when?**
> - **Data room (window or browser):** training, presentation, tutorials, tests of the portfolio chain — everything that needs a whole situation picture.
> - **Single ART:** one Jira export for transform_data and build_reports, targeted flow patterns for cycle-time scatterplot training, continuation of an earlier extract.

This manual describes the data room first (chapters 2 to 7) and the classic single-ART generator afterwards (chapter 8). If you only want the one minute to the first portfolio report, take the tutorial “Demo data room in one minute” (docs/tutorials/demo-data-room.md, also as PDF).

> **Typical uses:**
> - **Test a new installation:** does the chain run from raw data to the portfolio report?
> - **Demos and presentations:** a situation with outlier, weak source and seams, with no privacy question
> - **Training:** the same story for every participant, reproducible through the seed
> - **Development:** stress test with a large structure (scale L, up to 49 ARTs)

Status: beta, October 2026. The data-room UI is new; the tab for per-ART profiles in the UI is still missing (command line: --art-profiles).

2 Getting started

2.1 Portable package (recommended)

The portable package embeds Python — nothing to install. After unpacking, four start files for the generator are ready:

Start file (Windows)	What it opens
`DatenraumFenster.bat`	**Data room as a window** — the default path. Three steps, no network port, runs on company computers too.
`SituationReport.bat`	The launcher with all modules; it carries the cards “Data room (window)” and “Data room (browser)” as well as “Testdata Generator” for the single ART.
`DatenraumBrowser.bat`	**Data room in the browser** — the same three steps as a web page. Starts a local server on 127.0.0.1 with a random key in the link. Private computers only: security software on company computers blocks and reports local servers.
`TestdataGenerator.bat`	**Single ART** — the classic window with workflow file, knobs and flow patterns (chapter 8).

macOS: the same names as .command (right-click → Open). Linux: as .sh (python3 and python3-tk required). The first start sets up a Python environment once — this needs internet.

Note for Windows: if the start reports that Python cannot be found, Windows had blocked the ZIP file. Right-click the ZIP → Properties → Security → Unblock → OK → unpack again.

2.2 From source (Python required)

The three UIs correspond to three calls: window, browser, single ART.

```
python -m testdata_generator --dataspace
python -m testdata_generator --web
python -m testdata_generator
```

2.3 Window or browser?

Both show the same three steps with the same settings and produce the same data room. The window is an ordinary program window (tkinter) and needs nothing else. The browser page is slightly more comfortable to read but runs through a local server — on a company computer the endpoint security blocked and reported that in the first test. Hence: window first, browser only privately.

2.4 Languages

UI, story, README, progress, log and the generated delta briefings exist in German, English, Romanian, Portuguese and French. The generator starts in the language set in the launcher; the switch at the top right of window and browser page changes it — inputs are kept. One run has one language: UI and generated texts match. Without a launcher setting and without `--lang` the language is English. Romanian, Portuguese and French are drafts still to be reviewed by native speakers.

3 The data room in three steps

Window and browser page lead through three steps: **Scenario** (which situation), **Scope & sources** (how big, how long, how real) and **Generate** (where to, and go). The defaults produce the demo situation that the tutorials and manuals of the portfolio modules use as well.

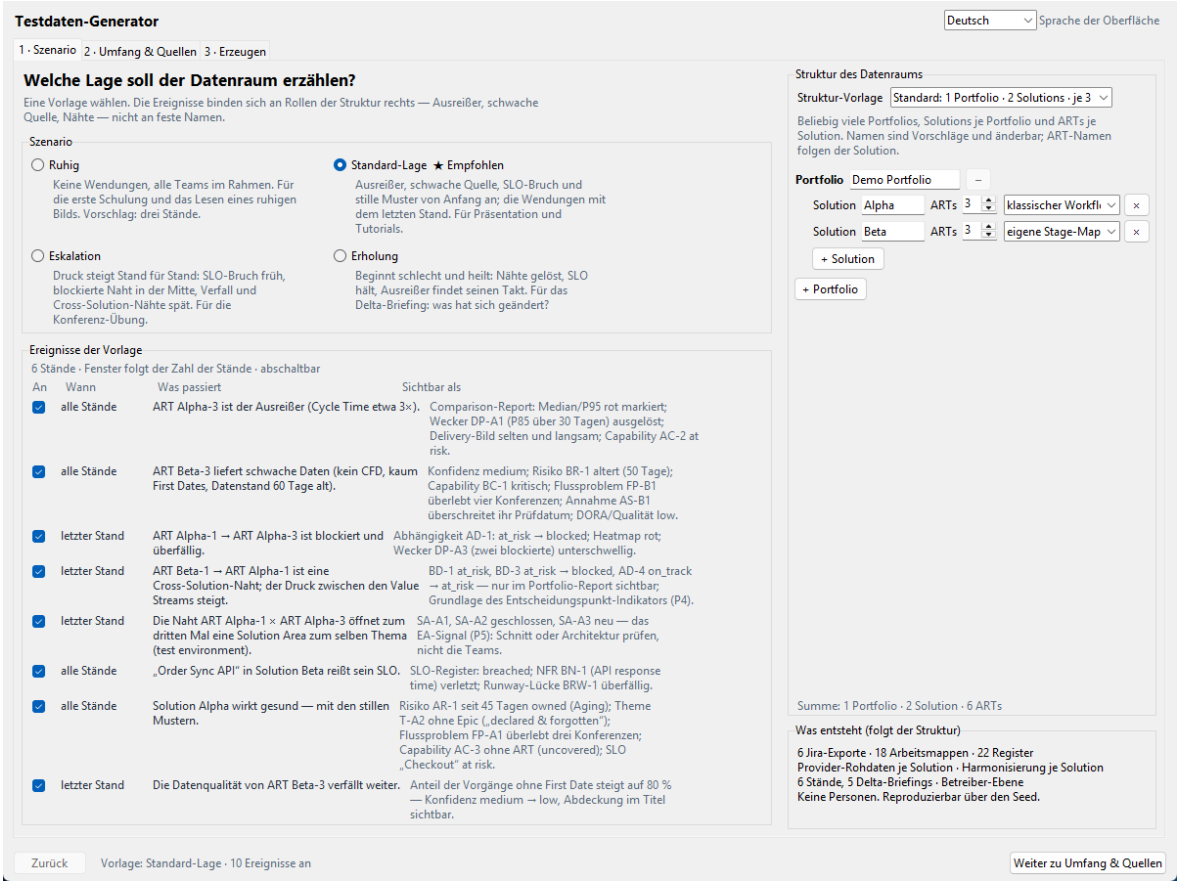


Fig. 1: Step 1 "Scenario" in the data-room window — templates on the left, events below, structure on the right (German UI)

3.1 Step 1 — Scenario

On the left you pick a **template** of the story. The events bind to roles of the structure on the right — outlier, weak source, seams — not to fixed names.

Template	What it tells	Stands
Calm	No turning points, every team within bounds. For the first training and for reading a quiet picture. Whoever raises an alarm here has misread the picture.	3
Standard situation (recommended)	Outlier, weak source, SLO breach and silent patterns from the start; the turning points arrive with the last stand. For presentations and tutorials.	6
Escalation	Pressure rises stand by stand: SLO breach early, blocked seam in the middle, decay and cross-solution seams late. For the conference exercise.	6

Template	What it tells	Stands
Recovery	Starts badly and heals: seams resolved, SLO holds, the outlier finds its rhythm. For the delta briefing: what has changed?	6

Events of the template. The table below shows for each event when it applies (all stands, from stand n, last stand), what happens and as what it becomes visible in the report. Every event has a checkbox; switched-off events are absent from the data room. An event whose prerequisite is off stays off with a notice. The window of an event follows the number of stands from step 2. Chapter 5 explains the ten events.

Structure of the data room (right). A structure template — Small (1 portfolio · 1 solution · 2 ARTs), Standard (1 · 2 · 3 each), Large (5 portfolios · 3–4 solutions · 1–6 ARTs) — or your own: add or remove portfolios and solutions, count ARTs per solution, change names (ART names follow the solution). Per solution you choose the classic workflow or an own stage map. At least one ART per solution.

What is produced. The box at the bottom right keeps count: Jira exports, workbooks, registers, stands and delta briefings for the chosen structure (standard: 6 exports, 18 workbooks, 22 registers, 6 stands, 5 deltas). From 30 ARTs it warns: generation then takes noticeably longer.

3.2 Step 2 — Scope & sources

The defaults are enough for training and presentation.

Setting	Default	Meaning
Stands	6	2 to 12 stands, 14 days apart; every stand is a frozen snapshot, every consecutive pair a rendered delta briefing. The “Calm” template suggests 3.
Scale	M	S = story anchors only, short registers; M = realistic registers; L = stress test with full registers (dense heatmap, full roadmap matrix). The anchors and their stories are identical at every level.
Realism	60	Untidiness of the status paths: skips, cancels, parking, loops, reopening. 0 = smooth, 100 = like real; 60 is a typical ART.
Seed	42	The same seed, structure and reference date yield the same data room — ideal when every participant should see the same thing.
Last stand	today	Date of the last stand; the others lie 14 days apart before it. The quality traffic light of the portfolio report thus rates the data as current.
Language of the texts	UI language	Language of story, README and delta briefings; follows the UI.

Details for analysts. Cycle times, flow patterns and PI length can be overridden per ART — the same knobs as for the single ART. In this beta that works on the command line with `--art-profiles FILE.json` (chapter 6); the table in the UI follows with the ART tab.

Sources shipped with the room. Always included: Jira exports per ART (no persons), Prometheus SLO responses, GitHub deployments/PRs/incidents for the first solution, GitLab DORA daily values for

the second, SonarQube quality measures and an SLO CSV for the manual path — raw data in each system's own format with the same numbers as the registers, so that `sources fetch` runs in the data room without network.

AI drafts (switch, off by default). A mock spoken briefing, three answers, a prompt bundle and a practice answer with wrong numbers land in ai_drafts/ — all carrying the AI marker, none approved. With them you practise approval and `llm check` without installing a model.

The **Estimate** line at the end names Jira exports, registers, stands and deltas for your settings; the footer summarises template, scale, stands, realism and seed.

3.3 Step 3 — Generate

Target folder. It is created. If it is not empty, the generator asks — “Generate anyway” overwrites files of the same name, “Choose another folder” goes back. “Browse …” opens the folder dialog; where none is available, you type the path.

Progress. “Generate data room” starts nine steps; each shows running, done or skipped with a detail. Duration ranges from seconds to minutes depending on scope (standard situation about 10 seconds).

Step	What is produced
Structure and story	structure.json and story.json: roles bound, events fixed
Jira exports and workbooks	per ART one Jira export under raw/ and IssueTimes/Transitions/CFD under solutions/…/arts/
Registers	per solution the registers (risks, dependencies, NFR, SLO, DORA, capabilities, themes …)
Configurations	portfolio.json, solution.json per solution, pi_config.json
Provider raw data	sources/<solution>/ with replay files and sources configs
README	README.md with structure, story, stands and further paths
Series of stands	snapshots/snapshot_<nn>_<date>.json per stand, stands.json, prev/now
Delta briefings	deltas/delta_<nn>-<nn+1>.md and .html per consecutive pair
AI drafts (mock)	ai_drafts/ — only with the switch, otherwise “skipped”

Cancel stops after the running step; the folder contains what was built until then. An error appears with its message in the list.

Result card. After the last step “Data room ready” appears with duration and seed, four figures (issues and ARTs, registers and solutions, stands, source types), the story in three sentences and buttons: Open portfolio report, Conference folder, Latest delta, README, Open folder, AI drafts (if generated). Reports are rendered on the first click and opened in the browser.

Compare delta. Pick two stands (say 3 → 4) and “Open delta briefing” — all consecutive pairs lie ready under deltas/.

Afterwards. “New data room with another template …” jumps back to step 1; the line below shows the command for continuation (chapter 7).

4 What lies in the folder

The target folder is a **portfolio data room**: all paths in the configs are relative to their config file. The folder can be moved, copied or zipped as a whole.

Path	Content
`structure.json`	Structure as data: portfolios → solutions → ARTs, workflow per solution
`story.json`	Story as data: template, bound roles, events with switch and stand window, notices
`README.md`	Description of structure, story, stands, sources and further paths — in the language of the run
`portfolio.json`	Entry point: portfolio config across all solutions
`pi_config.json`	PI intervals across the data period
`solutions/<name>/`	per solution solution.json (own stage_map where chosen), arts/ with IssueTimes, Transitions and CFD per ART, registers/ with the registers under standard names
`raw/`	Jira exports per ART in REST API format — at the same time demo input for get_data
`sources/<name>/`	provider raw data as replay files (Prometheus, GitHub or GitLab, SonarQube, CSV) plus sources configs
`snapshots/`	snapshot_<nn>_<date>.json per stand, index stands.json, snapshot_prev/now.json = the last two
`deltas/`	delta_<nn>-<nn+1>.md and .html per consecutive pair — rendered
`ai_drafts/`	only with the switch: mock spoken briefing, answers, prompt bundle, practice answer — all with the AI marker

The folder is directly usable — portfolio report, delta briefing of any pair, conference folder, `sources fetch` without network on the replay files (in sources/<solution>/) and the export check of get_data:

```
python -m portfolio portfolio.json
python -m portfolio --delta snapshots/snapshot_prev.json snapshots/snapshot_now.json
python -m portfolio portfolio.json --conference mappe.html
python -m sources fetch
python -m get_data check raw/<art>_jira.json
```

With several portfolios (template “Large” or own structure) each portfolio becomes its own data room in a sub-folder; a README in the main folder points to them. A situation picture across several portfolios is not a feature today — the data for it already lies side by side.

5 Understanding the story

Every template tells the same situation from all registers: risks, dependencies, assumptions, SLOs and alarms point at the same places. The events bind to **roles** that follow from the structure — so the story stays coherent with your own structure too.

> **The roles:**
> - **First solution** (“host”): looks healthy, carries the silent patterns; its **last ART is the outlier**; between its ARTs lie the blocked dependency and the recurring seam.
> - **Second solution:** its **last ART is the weak data source**; here the “Order Sync API” breaches its SLO and the new security risk appears.
> - **Cross-solution seam:** the first ART of the second solution depends on the first ART of the first — visible only in the portfolio report.
> - Further solutions and portfolios receive the seed-generated base set without story anchors.

What the structure cannot provide is dropped with a **notice** (in step 1 and in story.json): with a single solution there is no weak source, no SLO breach and no cross-solution seam; if the first solution has only one ART, outlier, blocked dependency and recurring seam are missing. The ten events:

No.	What happens	Visible as
E1	The outlier ART has about three times the cycle time.	Comparison report: median/P95 red; alarm “P85 over 30 days” triggered; delivery picture rare and slow; one capability at risk.
E2	The weak source delivers weak data: no CFD, hardly any first dates, data 60 days old.	Confidence medium; one risk ages (50 days); one capability critical; one flow problem survives four conferences; one assumption passes its review date; DORA/quality low.
E3	The dependency between two ARTs of the first solution is blocked and overdue.	Dependency at_risk → blocked; heatmap red; alarm “two blocked” just below threshold.
E4	The cross-solution seam comes under pressure; the value streams pull at each other.	Several dependencies slide from on_track via at_risk to blocked — visible only in the portfolio report; basis of the decision-point indicator.
E5	The same seam opens a solution area on the same topic (test environment) for the third time.	Two solution areas closed, one new — the EA signal: check the cut or the architecture, not the teams.
E6	The “Order Sync API” of the second solution breaches its SLO.	SLO register: breached; NFR “API response time” violated; one runway gap overdue.
E7	The first solution looks healthy — with the silent patterns.	One risk owned for 45 days (aging); one theme without epic (“declared & forgotten”); one flow problem survives three conferences; one capability without ART; SLO “Checkout” at risk.
E8	The data quality of the weak source decays further.	Share of issues without first date rises to 80 % — confidence medium → low, coverage visible in the title.

No.	What happens	Visible as
E9	New risk in the second solution: security review not scheduled.	A high, mitigated risk appears — new in the delta briefing.
E10	Roadmaps updated.	One epic loses its theme (zombie initiative); one epic of the second solution is pulled from P2 to P1.

Stand windows. Every template fixes in which stands an event applies. Standard situation: outlier, weak source, SLO breach and silent patterns in all stands, the turning points (blockage, seams, decay, new risk, roadmaps) only in the last. Escalation staggers them over the series, Recovery reverses them: stand 1 shows everything, then one thing after another heals. The README of the data room lists each event's window; before and after it the series shows the situation without it.

6 The data room on the command line

Everything from the three steps is also an option of `python -m testdata_generator --scenario portfolio`. Before building, the command line prints structure, switches and estimate; afterwards folder, stands and duration.

Option	Default	Meaning
--scenario portfolio	—	Generate a data room instead of a single ART
--output FOLDER	(required)	Target folder (is created)
--structure small|standard|large|FILE.json	standard	Structure template or own structure.json (portfolios → solutions → ARTs, workflow per solution)
--arts N[,N...]	—	One portfolio from ART counts per solution, e.g. 3,4s,1 — three solutions with 3, 4 and 1 ARTs; a trailing s gives that solution its own stage map. Not combinable with --structure.
--story calm|standard|escalation|recovery	standard	Story template (calm, standard, escalation, recovery)
--event ID=on|off	—	Switch one event on or off, e.g. --event E3=off --event E9=on; repeatable. An event whose prerequisite is off stays off with a notice.
--stands N	6 (calm 3)	Number of stands (2–12), 14 days apart, the last on the reference date; every consecutive pair gets a delta briefing under deltas/
--scale s|m|l	m	Register volume per solution: s = anchors only, m = realistic demo, l = stress test
--seed N	(random)	Seed for reproducible output; the UI uses 42
--art-profiles FILE.json	—	Knobs per ART as JSON, e.g. {"Alpha-1": {"mean_cycle_days": 30, "pattern": "cluster", "pattern_strength": 80}}. Fields: issue_count, mean_cycle_days, std_cycle_days, completion_rate, todo_rate, backflow_prob, pattern, pattern_strength, pi_duration_weeks, realism. Overrides apply in all stands; everything else keeps its story value.
--ai-drafts	off	Write mock AI drafts with marker to ai_drafts/
--lang de|en|ro|pt|fr	suite language, else en	Language of UI, console and generated texts
--continue FOLDER --to-date DATE	—	Continue an existing data room up to the date (chapter 7)
--web / --port N / --no-browser	—	Start the browser UI: local server on 127.0.0.1 (port 0 = random free), access key in the link; --no-browser only prints the link. Ends with Ctrl+C.

Option	Default	Meaning
--dataspace	—	Start the data-room window — the same three steps without a server

Examples

```
# Standard situation, as in tutorial and manuals
python -m testdata_generator --scenario portfolio --output demo/ --seed 42

# Escalation over eight stands, stress test, German texts
python -m testdata_generator --scenario portfolio --output demo-esc/ \
    --story escalation --stands 8 --scale l --lang de

# Own structure, one event off, AI practice on
python -m testdata_generator --scenario portfolio --output demo-3/ \
    --arts 3,4s,1 --event E3=off --ai-drafts

# Five portfolios, 49 ARTs — takes noticeably longer
python -m testdata_generator --scenario portfolio --structure large \
    --output demo-large/
```

Without --lang and without a launcher setting, console and generated texts are English. For German pass --lang de or switch the launcher.

7 Continuing a data room

A data room can be continued to a later date — like a second extract from Jira: the past stays, open issues move on, new ones are added.

```
python -m testdata_generator --scenario portfolio \
    --continue demo/ --to-date 2027-01-15
```

The generator continues the Jira exports, IssueTimes/Transitions/CFD, configs and the PI config in place; the previous snapshot_now becomes snapshot_prev, the new stand snapshot_now, and the README gets a note with old and new reference date and the number of new issues. The registers stay unchanged. The knobs come from the generator record of the files; the language follows `--lang`.

The result card shows the matching command under “Afterwards”. For a single Jira export there is --continue OLD.json (chapter 8).

8 A single ART (classic generator)

The classic path produces **one** Jira export file in REST API format from a workflow file: issues pass through the stages, dwell for varying times, occasionally fall back and close at a configurable rate. The file goes straight into transform_data and build_reports. The knobs for cycle time and flow patterns are the same ones the data room overrides per ART through --art-profiles.

8.1 The window

TestdataGenerator.bat (or `python -m testdata_generator` without options) opens the window with input fields for every parameter. Only the workflow file is mandatory; everything else has defaults. The “Demo portfolio” area at the bottom builds the standard situation at a click — the data room of chapter 3 can do more.

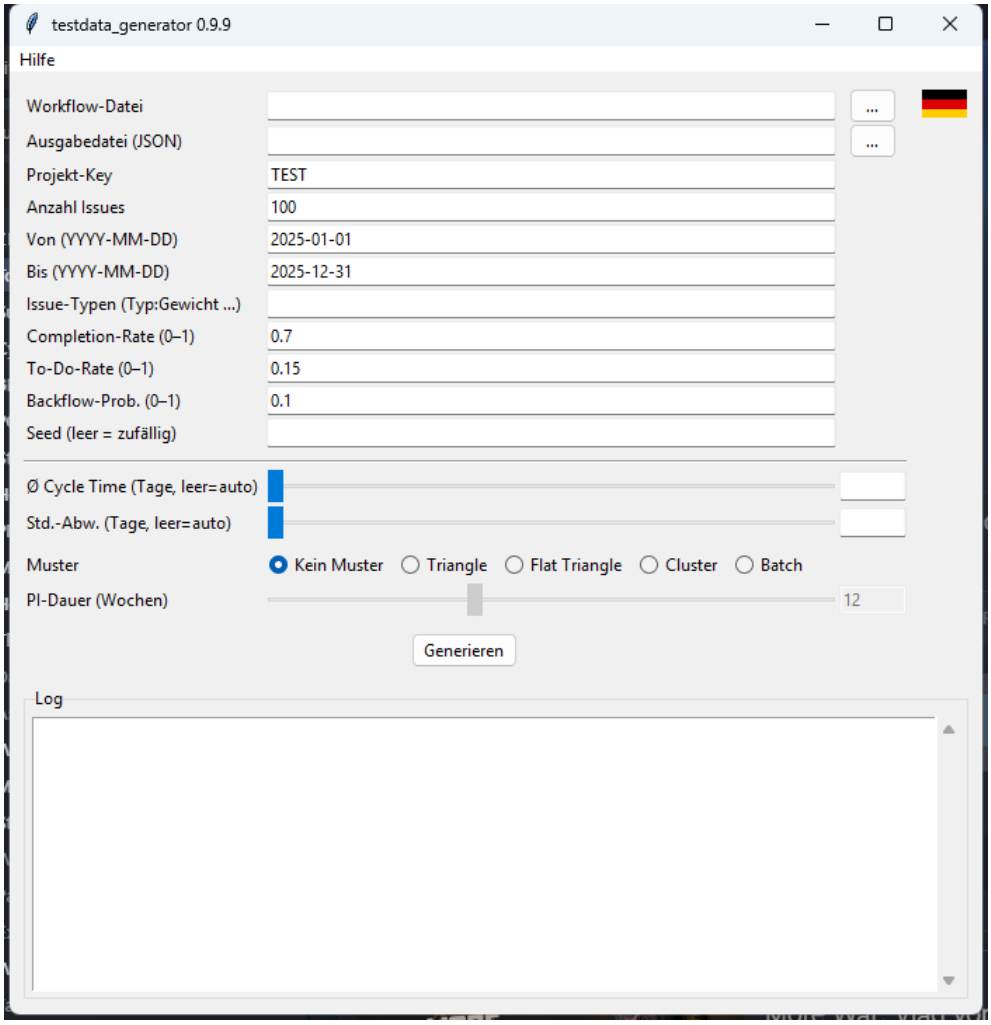


Fig. 2: Window of the classic generator for a single ART

8.2 The workflow file

The workflow file defines the stages of your Jira board and is the only mandatory field. It has the same format as in transform_data — one file serves both modules. Each line describes a stage; aliases (alternative status names in the Jira export) are separated by colons. Two special lines mark the start and end of active work:

```
CanonicalName:Alias1:Alias2
<First>FirstActiveStage
<Closed>ClosedStage
```

Line	Meaning
Name:Alias1:Alias2	Stage with one or more alternative Jira status names
Name (no :)	Stage without aliases
<First>Name	First active stage — cycle time starts here
<Closed>Name	Closed stage — marks an issue as done

The bundled example `workflow_ART_A.txt` models a typical SAFe ART workflow:

```
Funnel:New:Open:To Do
Analysis:In Analysis:Estimated
Program Backlog:Ready for Dev:Backlog
Implementation:In Implementation:In Review:In Progress
Blocker
Validating on Staging:QA
Deploying to Production:Ready for Development
Releasing:Completed
Done:Canceled
<First>Analysis
<Closed>Releasing
```

“Funnel” has four aliases — all four Jira statuses normalise to Funnel. Analysis is the first active stage (cycle time starts here). Releasing counts as closed; Done (after Releasing) also takes cancelled issues.

8.3 Parameter reference

All parameters exist in the window and on the command line.

Parameter	Default	Description
--workflow FILE	(required)	Workflow definition file (.txt)
--output FILE.json	_generated.json	Output file path
--project KEY	TEST	Jira project key of the generated issues
--issues N	100	Number of issues
--from-date YYYY-MM-DD	2025-01-01	Earliest creation date
--to-date YYYY-MM-DD	2025-12-31	Latest transition date
--issue-types TYPE:W ...	Feature:0.6 Bug:0.3 Enabler:0.1	Issue types with weights (need not sum to 1)
--completion-rate FLOAT	0.7	Share of issues reaching the closed stage (0–1)
--todo-rate FLOAT	0.15	Share of open issues staying in early stages (0–1)
--backflow-prob FLOAT	0.1	Probability of a backward step per transition (0–1)
--realism 0-100	0	Untidiness of the status paths: skips, cancels, parking, loops, reopening; about 70 is like a real ART
--seed INT	(random)	Seed for reproducible output
--mean-cycle-days FLOAT	—	Mean cycle time in days (lognormal); enables cycle-time control
--std-cycle-days FLOAT	30 % of mean	Standard deviation of cycle time in days
--pattern NAME	none	Flow pattern: none / triangle / flat_triangle / cluster / batch (see 8.4)
--pi-duration-weeks INT	12	PI length in weeks (for cluster and batch)
--pattern-strength 0-100	50	Pattern intensity (0 = subtle, 50 = default, 100 = strong)
--continue OLD.json	—	Continue an earlier extract up to --to-date: the past stays byte for byte, open issues move on, new ones are added; needs --workflow
--learn-from FILE.xlsx	—	Learn a transition profile from a real transitions file (counts and hours only); --transition-profile generates with it

8.4 Flow patterns

The generator simulates typical flow anti-patterns from the literature of Vacanti and Singh. Combine --mean-cycle-days with --pattern; the window offers radio buttons and sliders.

Pattern	Scatterplot	Description
none	Even point cloud	Random cycle time without trend — default behaviour
triangle	Triangle, right angle bottom right	Cycle time rises linearly over time (about 1–4× at default strength). A process that slows down more and more.
flat_triangle	Triangle, rise flattens	Like triangle, the rise flattens towards the end. A deterioration that stabilises.
cluster	Vertical clusters at PI end	Cycle time stays stable, but most issues close in the last two weeks before PI end. Deadline-driven behaviour.
batch	Columns of varying height at PI end	Like cluster, but with strongly varying cycle time (0.1× to 3× mean). Short and very long items are pushed to the PI end together.

--pattern requires --mean-cycle-days; without it the pattern is ignored. --std-cycle-days controls the spread, --pi-duration-weeks the PI length for cluster and batch.

8.5 ART_A — complete example

From the generator to the finished report on the fictional project ART_A:

Step 1: Generate test data

```
python -m testdata_generator \
    --workflow testdata_generator/workflow_ART_A.txt \
    --project ART_A --issues 200 \
    --from-date 2025-01-01 --to-date 2025-12-31 \
    --issue-types Feature:0.6 Bug:0.2 Enabler:0.2 \
    --completion-rate 0.75 --backflow-prob 0.08 \
    --seed 42 --output ART_A_generated.json
```

Step 2: Process with transform_data

```
python -m transform_data ART_A_generated.json \
    --workflow testdata_generator/workflow_ART_A.txt
```

Step 3: Create the report with build_reports

```
python -m build_reports
```

What to expect (seed 42, 200 issues):
- Keys ART_A-0001 to ART_A-0200
- Types about 120 Feature, 40 Bug, 40 Enabler
- About 150 issues in Releasing or Done
- Creation dates spread over 2025
- Occasional backward steps (backflow-prob 0.08 = about 8 %)

9 Frequently asked questions

Window or browser — which should I use?

The window. It produces the same data room and opens no network port. Use the browser page only on a private computer; on company computers the security software blocks and reports local servers.

Why is my data room in English although I speak German?

Without a language argument the generator takes the suite language from the launcher, else English. Set the launcher to German, switch at the top right or pass --lang de.

The target folder is not empty — what happens?

The generator asks. “Generate anyway” overwrites files of the same name and leaves everything else; “Choose another folder” aborts.

Generation takes long.

From 30 ARTs, or at scale L with several portfolios, the “What is produced” box warns. For training, standard structure and scale M are enough (about 10 seconds).

Test data or real data?

For demos, training and installation tests, test data is the better choice — no privacy question. Real flow analyses and decisions need real data; the manual export is described in the Get Data manual.

Can I set individual ARTs differently?

Yes, through --art-profiles FILE.json on the command line (chapter 6). The tab for it in the UI follows in a later beta.

10 Glossary

Term	Explanation
ART	Agile Release Train — several teams delivering together in SAFe
Data room	A folder with everything a portfolio situation picture needs: configs, exports, registers, stands, briefings — self-contained and movable
Delta briefing	“What has changed?” between two stands — throughput, confidence, seams, new risks
Conference folder	The folder for the value stream conference: flow problems that survive conferences, with the situation picture
Seam	A dependency between ARTs or solutions where pressure shows — within a solution or between value streams
Register	A list per solution: risks, dependencies, assumptions, NFR, runway, SLO, DORA, capabilities, themes, flow problems …
Stand	A frozen snapshot of the situation picture at a date; stands lie 14 days apart
Seed	Start value of the random generator — same seed, same output
Solution	A group of ARTs delivering one solution together; in the data room the level with its own registers
Stage map	A solution's own grouping of the workflow stages (say lead-in/implementation/done) instead of the default path
Value stream	The flow from need to value; represented in the data room by the solutions
Workflow	Ordered sequence of stages an issue passes through; described as a text file
Realism	Degree of untidiness in the status paths: skips, cancels, parking, loops, reopening
CFD	Cumulative Flow Diagram — stock per stage over time, cumulated